

Explicación number-of-strings

Juan Yago Castells

September 12, 2026

Explicación detallada sobre por qué funcionan los métodos utilizados y cuáles fueron las razones que determinaron su uso en este proyecto. Sin un fin explicativo extenso, tan solo indagaré en la intuición de los algoritmos.

§1 Conteo del número de palabras de longitud n

Para dar el número de palabras de una determinada longitud utilizo la representación formal del AFD aprovechando la estructura de grafo que tiene y evitando utilizar la expresión regular resultante del autómata que puede contener redundancias por cómo está construida. Además de que trabajar con un AF determinista ofrece una gran ventaja y es que en el j -ésimo paso nos asegura que solo ha podido hacer j saltos de estado, posiblemente repitiendo alguno y que se encuentra en un solo estado a la vez, no como los AF no deterministas.

§1.1 Uso de la multiplicación matricial

Aprovechando las ventajas de la teoría de grafos que ofrece la representación formal del AFD y su determinismo llegamos a que una de las mejores formas de computar el número que buscamos es mediante la exponenciación de una matriz. Para entender esto vamos a ver qué representa esta matriz que llamaremos M . Cada entrada de M denotada como M_{ij} representa el número de transiciones (definidas en δ) que llevan del estado q_i a q_j , en otras palabras, consumiendo un único símbolo el autómata puede ir de q_i a q_j de M_{ij} formas distintas. Ahora veamos qué pasa cuando elevamos M a una potencia, por ejemplo M^2 , para poner un ejemplo supongamos que el autómata A tiene 3 estados $Q = \{q_1, q_2, q_3\}$, por tanto tendremos que M es de dimensión 3×3 :

$$M = \begin{pmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{pmatrix}$$

Cada entrada t_{ij} representa las transiciones que hemos explicado, ahora si elevamos M al cuadrado veamos en qué se transforma la entrada t_{11} :

$$t_{11} = t_{11}t_{11} + t_{12}t_{21} + t_{13}t_{31}$$

Resultado obtenido mediante la multiplicación de la primera fila por la primera columna. Ahora analicemos esto y generalicemos. La casilla que representaba las transiciones con un solo símbolo desde q_1 hasta q_1 ahora incluye también todas las que se quedan en q_1 así como las que iban a q_2 y ahora vuelven a q_1 y lo mismo para q_3 , es decir, ahora t_{11}

contiene todas las formas posibles de quedarse en el primer estado cuando A consume 2 símbolos. En general para M^k la entrada $(M^k)_{ij}$ almacenará

$$\sum_{n=1}^k \left(\sum_{x=1}^n t_{ix} t_{xj} \right)$$

La suma interior representa todas las combinaciones posibles de caminos de n símbolos y la exterior acumula estas combinaciones para todas las longitudes de caminos desde 1 hasta k .

§2 Minimización del ADF

Junto a la misma definición de autómata en el archivo `AFD.hs` tenemos la función `simpHopcroft` que implementa en lenguaje funcional el mismo algoritmo que muestra el libro.

Tiene un tiempo de ejecución de $O(nk \log n)$ siendo n el número de estados del autómata finito determinista original y k la cantidad de símbolos en su alfabeto, aunque como este número suele ser una constante pequeña, sobre todo si solemos trabajar con el alfabeto binario, entonces se suele mostrar este algoritmo como de complejidad $O(n \log n)$.

§2.1 Implementación en un lenguaje funcional (Haskell)

El código en Haskell traduce directamente la implementación en pseudocódigo de [Xu \(2009\)](#) que se basa en el artículo original de [Hopcroft \(1971\)](#). La explicación es sencilla, asumo un mínimo conocimiento de Haskell y no pretendo ser exhaustivo.

La función `simpHopcroft` recibe un AFD y produce un conjunto de conjuntos de estados equivalentes, llamémoslo Q' . Lo primero que hace es computar un *Map* que será la función delta inversa (δ^{-1}) como un Mapa que relaciona una clave (*Estados*, *Caracter*) con un conjunto S , de forma que en $O(1)$ podremos saber qué estados llegan a alguno de *Estados* leyendo *Caracter*.

La función `go` se encarga de iterar sobre todos los conjuntos dentro de W y sobre los símbolos del alfabeto, utiliza `foldl` `ptfset` ra modificar P y W tal y como se describe en algoritmo de Xu.

Por último y sin ánimo de completitud la función `procesar` es la encargada de ejecutar el bucle más interno, también con `foldl` para iterar sobre los conjuntos R de P candidatos a ser divididos, notar que la condición $R \cap l_a \neq \emptyset$ y $R \not\subseteq l_a$ es igual a la que utilizo en el código $R \cap l_a \neq \emptyset$ y $R - (R \cap l_a) \neq \emptyset$.

§2.2 Complejidad temporal de la implementación

En este momento falta aclarar la motivación detrás del uso de las estructuras `Data.Map`, `Data.Set` y `Data.List` de Haskell. La idea es no alejarnos demasiado del excelente límite asintótico $n \log n$ que nos da Hopcroft. Para ello es vital computar δ^{-1} una única vez y que sus consultas sean en tiempo constante, por eso he utilizado un `Map`. Para editar los conjuntos P y W es vital que el tiempo no sea lineal ya que procesar el conjunto de estados de tamaño n y en cada iteración hacer una búsqueda lineal de $O(n)$ nos dejaría

con un algoritmo $O(n^2)$ destrozando toda eficiencia, por ello los **Set** que ofrecen búsqueda y eliminación en tiempo logarítmico son esenciales para no aumentar la complejidad asintótica.

Referencias

- John Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In Z. Kohavi and A. Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, New York, 1971. URL <http://i.stanford.edu/pub/cstr/reports/cs/tr/71/190/CS-TR-71-190.pdf>.
- Yingjie Xu. Minimizing states. In *Describing an $n \log n$ Algorithm for Minimizing States in Deterministic Finite Automaton*, 2009. URL <https://api.semanticscholar.org/CorpusID:14461400>.